

PROJECT 6 — PHASE 9, TASK 1

Sensitive Action Step-Up Detection

Complete Command-by-Command Reference & Explanation

Every command run, every file edited, and why — including the two bugs found and fixed along the way.

Vignesh Gopal Kalaivani

1. What This Task Achieves

Task 1 detects a specific gap in the original ATO detection (Phases 1-8): once a session is authenticated, it was trusted indefinitely. A session left open for a long time, or a stolen/replayed session, could still be used to perform a sensitive action (like a fund transfer) with no additional verification.

The fix: sensitive actions now check how long ago the session logged in. Inside a short window, the action proceeds normally. Outside that window, the action is blocked and logged as suspicious — mirroring real banking UX, where browsing is low-friction but a transfer re-prompts for a password or OTP regardless of how long you've been logged in.

2. Step-by-Step Commands and Explanations

Step 1 — Add session handling and login-time tracking to login.php

```
sudo nano /var/www/html/securebank/login.php
```

The original login.php only checked credentials and returned HTTP 200/401 — it never created a real session. Since a "session" is the exact thing that gets abused in this scenario, session support had to be added first. Three lines were added inside the success branch:

```
session_start();                                // at the top of the file
session_regenerate_id(true);
$_SESSION['user'] = $user;
$_SESSION['login_time'] = time();
```

- `session_start()` — must be the very first statement in the file; creates or resumes a PHP session and makes `$_SESSION` usable.
- `session_regenerate_id(true)` — issues a brand-new session ID specifically at the moment of login, rather than reusing whatever ID existed beforehand. This is a real security best practice (prevents session fixation attacks).
- `$_SESSION['user']` — records which account this session belongs to, so `transfer.php` can check who is logged in.
- `$_SESSION['login_time'] = time()` — records the exact Unix timestamp (seconds since 1970) the session was created. This is the reference point Task 1's entire step-up logic depends on.

Step 2 — Create the log directory

```
sudo mkdir -p /var/log/securebank
sudo chown www-data:www-data /var/log/securebank
```

Apache and PHP run as the `www-data` user on Debian. Without ownership of this directory, every attempt to write a log file from PHP would silently fail with a permissions error, and no evidence of the new detection signal would ever appear.

```
ls -la /var/log/ | grep securebank
```

Used to confirm the directory was created with the correct owner before moving on — verifying each step before proceeding is the same discipline used throughout every earlier phase of this project.

Step 3 — Create transfer.php with the session-age check

```
sudo nano /var/www/html/securebank/transfer.php
```

This file simulates a sensitive banking action (a fund transfer) and is the actual enforcement point for the step-up logic:

```
$STEPUP_WINDOW_SECONDS = 120;
$page = time() - ($_SESSION['login_time'] ?? 0);
if ($page > $STEPUP_WINDOW_SECONDS) {
    // log SENSITIVE_ACTION_NO_STEPUP, return HTTP 403
} else {
    // log SESSION_ACTION, allow the transfer
}
```

- `$STEPUP_WINDOW_SECONDS = 120` — the threshold: sensitive actions are trusted without extra verification only within 2 minutes of login. Set short deliberately, purely so both outcomes could be tested quickly in a lab setting rather than waiting a realistic amount of time.
- `$age = time() - ($_SESSION['login_time'] ?? 0)` — calculates how many seconds have passed since login. The `?? 0` fallback handles the edge case where `login_time` somehow was never set, defaulting to a large calculated age so the code safely falls into the 'blocked' branch instead of crashing.
- If `$age` is too large: HTTP 403 (Forbidden) is returned, and the event is logged as `SENSITIVE_ACTION_NO_STEPUP` — this is the new detection signature this task exists to create.
- If `$age` is within the window: the transfer proceeds normally, logged as `SESSION_ACTION` — representing legitimate behavior.
- `error_log(..., 3, "/var/log/securebank/session.log")` — the 3 tells PHP to append the message to a specific file rather than the system's default log, which is why the writable directory from Step 2 was required first.

Step 4 — Test both scenarios manually with curl

Test A — action within the window (expected to succeed):

```
curl -c cookies.txt -X POST http://192.168.1.30/securebank/login.php -d
"username=jsmith&password=Winter2026!"
curl -b cookies.txt http://192.168.1.30/securebank/transfer.php
```

- `-c cookies.txt` — tells curl to save the session cookie returned by `login.php` into a file, so it can be reused on the next request.
- `-b cookies.txt` — tells curl to send that saved cookie back, simulating the same browser session continuing.

Result: "Transfer of \$500 initiated for jsmith" — confirms the quick-action path works correctly.

Test B — action after the window (expected to be blocked):

```
curl -c cookies.txt -X POST http://192.168.1.30/securebank/login.php -d
"username=jsmith&password=Winter2026!"
sleep 130
curl -b cookies.txt http://192.168.1.30/securebank/transfer.php
```

- `sleep 130` — pauses the terminal for 130 seconds, deliberately exceeding the 120-second step-up window, so the next request should be treated as too old.

Result: "Additional verification required for this action." — confirms the delayed-action path is correctly blocked.

Step 5 — Confirm both events were logged correctly

```
sudo cat /var/log/securebank/session.log
```

Confirmed both `SESSION_ACTION` (legitimate, quick actions) and `SENSITIVE_ACTION_NO_STEPUP` (blocked, delayed action, with `session_age=256` shown) were written to the log with the correct fields.

3. Building the Custom Wazuh Decoder

`session.log` is a custom application log format, not a standard log type Wazuh recognizes out of the box (unlike Apache's `access.log`, which had a built-in decoder already). A custom decoder had to be written to teach Wazuh how to break this log format into structured, named fields.

Step 6 — Create the decoder file

```
sudo nano /var/ossec/etc/decoders/local_decoder.xml
```

Final, working version:

```
<decoder name="securebank-session">
  <prematch>^ATO LAB </prematch>
</decoder>
```

```
<decoder name="securebank-session-fields">
  <parent>securebank-session</parent>
  <regex type="pcre2" offset="after_parent">^(\S+) session_id=(\S+) user=(\S+).*?
  ip=(\S+)$</regex>
  <order>event_type, session_id, user, srcip</order>
</decoder>
```

- `<prematch>^ATO_LAB </prematch>` — the parent decoder's broad check: does this log line start with the literal text "ATO_LAB"? If yes, Wazuh tries the child decoder below.
- `<parent>securebank-session</parent>` — tells the child decoder to only run if the parent already matched.
- `type="pcre2"` — explicitly tells Wazuh to use the standard PCRE2 regex engine, rather than its own simplified OSRegex syntax. This was required after a real bug (explained below).
- `offset="after_parent"` — tells the regex to start matching from right after wherever the prematch left off, not from the beginning of the line again.
- The four capture groups (`\S+` = one or more non-whitespace characters) extract, in order: the event type, the session_id value, the username, and finally the IP address at the end of the line.
- `<order>event_type, session_id, user, srcip</order>` — assigns each captured group to a named field.

Step 7 — Validate with wazuh-logtest

```
sudo /var/ossec/bin/wazuh-logtest
```

Used repeatedly throughout this task to test the decoder and rule against a single pasted log line without needing to restart the live Wazuh service — the same safe validation discipline used in Phase 5.

4. Two Real Bugs Found and Fixed

Bug 1 — Invalid offset value

The very first version of the decoder used `offset="after_regex"`, which does not exist as a valid Wazuh offset option. Wazuh rejected it outright with a loading error. The only valid offset values are `after_parent` and `after_prematch`. Fixed by combining what had been two separate child decoders into a single one using `offset="after_parent"`.

Bug 2 — OSRegex does not support \S+ (silent failure, no error message)

After fixing Bug 1, the parent decoder matched correctly, but the child decoder extracted nothing at all — no error was thrown, it simply failed silently. Diagnosed by stripping the regex down to a trivially simple pattern (`^(.*)$`) and confirming the decoder mechanism itself worked, then rebuilding the pattern one capture group at a time using `wazuh-logtest` after each change.

Root cause: Wazuh's decoders use a custom pattern-matching engine (OSRegex) by default, which does not recognize standard regex character classes like `\S`. This is why the pattern appeared syntactically correct but extracted nothing. Fixed by explicitly adding `type="pcre2"` to the regex tag, switching to the full standard PCRE2 regex engine that does recognize `\S+`.

Bug 3 (behavioral quirk, not exactly a bug) — Reserved field name remapping

After fixing Bug 2, three of the four fields extracted correctly, but the field named `user` in the `<order>` tag was silently renamed to `dstuser` in Wazuh's actual output. This is because Wazuh treats certain field names (such as `user`) as reserved static fields with special platform-wide meaning, and automatically maps them to its own standardized name (`dstuser`, meaning destination user) rather than using the literal name provided. The extracted value was correct throughout — only the field name differed from what was specified.

Bug 4 — Log lines running together with no newline (the critical one)

After the decoder and rule both validated successfully in `wazuh-logtest`, the alert still did not fire against live, real traffic. Investigation of the raw `session.log` file revealed that all log lines had been written back-to-back with no line break between them, because the original `error_log()` calls did not include a trailing `\n` character.

Wazuh's log collector reads files line by line, so the entire file was being interpreted as one single giant line, and the `^ATO_LAB` prematch could only ever apply to the very first entry in the file.

Fixed by adding `\n` to the end of both `error_log()` message strings in `transfer.php`, then clearing the old malformed log file with:

```
sudo truncate -s 0 /var/log/securebank/session.log
```

After this fix, fresh test events were written as properly separated lines, and rule 100105 fired correctly against live traffic, confirmed directly in the alerts log.

5. Writing and Confirming the Detection Rule

```
sudo nano /var/ossec/etc/rules/local_rules.xml

<rule id="100105" level="10">
  <decoded_as>securebank-session</decoded_as>
  <field name="event_type">SENSITIVE_ACTION_NO_STEPUP</field>
  <description>Sensitive action performed without recent re-authentication (session
step-up missing)</description>
  <mitre>
    <id>T1078</id>
  </mitre>
  <group>attack,session_anomaly,</group>
</rule>
```

- `<decoded_as>securebank-session</decoded_as>` — matches by decoder name rather than a numeric rule ID, since this is a brand-new custom decoder with no existing built-in ID to chain off (unlike the 31108 approach used in Phase 5).
- `<field name="event_type">SENSITIVE_ACTION_NO_STEPUP</field>` — matches specifically on the decoded `event_type` field equalling this exact value, now that the decoder makes it available as a real structured field rather than raw text.

Final verification — confirming the alert fires live

```
curl -c cookies.txt -X POST http://192.168.1.30/securebank/login.php -d
"username=jsmith&password=Winter2026!"
sleep 130
curl -b cookies.txt http://192.168.1.30/securebank/transfer.php
sudo grep "Sensitive action performed" /var/ossec/logs/alerts/alerts.log
```

Result confirmed: Rule: 100105 (level 10) -> 'Sensitive action performed without recent re-authentication (session step-up missing)' appeared in the live alerts log, proving the entire chain — application logging, custom decoder, and custom rule — works correctly end to end against real traffic, not just isolated test lines.

6. Summary Table — All Commands Used

Step	Command / File Edited	Purpose
1	<code>nano login.php</code>	Add <code>session_start()</code> , <code>session_regenerate_id()</code> , <code>login_time</code> tracking
2	<code>mkdir + chown /var/log/securebank</code>	Create writable log directory for <code>www-data</code>
3	<code>nano transfer.php</code>	Add session-age check and step-up logic
4	<code>curl (with -c / -b cookies.txt)</code>	Manually test both the allowed and blocked scenarios
5	<code>cat session.log</code>	Confirm both event types logged correctly
6	<code>nano local_decoder.xml</code>	Build custom decoder to extract structured fields
7	<code>wazuh-logtest</code>	Validate decoder and rule safely before live deployment
8	<code>nano local_rules.xml</code>	Write rule 100105 against the new decoded fields
9	<code>truncate + retest</code>	Fix the newline bug and confirm live alert firing
10	<code>grep alerts.log</code>	Final confirmation of a live-fired alert

7. Key Lessons From This Task

- Custom decoders require explicit PCRE2 declaration to use standard regex syntax — Wazuh's default OSRegex engine silently fails on patterns it doesn't recognize, without raising an error.
- Certain field names (like user) are reserved by Wazuh and get automatically remapped (to dstuser) regardless of what name is specified in a custom decoder's <order> tag.
- A rule and decoder can both validate perfectly in isolated testing (wazuh-logtest) while still failing against live traffic — the root cause here was an application-logging detail (missing newlines) breaking the log collector's line-based parsing, not the rule logic itself.
- Systematic isolation (stripping a regex down to its simplest form and rebuilding one capture group at a time) is a far more reliable debugging method than repeatedly guessing at a full, complex pattern.